

Oblig 2

Algoritmer og datastrukturer

Andreas Isaksen

January 23, 2025

Oppgave 1

- a) Denne oppgaven er referert til med kommentarer i Java koden (Linje 27 og 93).
- b) Her er en liste over tester på metoden *superlineær*(*n*) med økende problemstørrelse(*n*).
- $n = 10 \rightarrow T(n \cdot \log_2) = 0ms$
 - $n = 100 \rightarrow T(n \cdot \log_2) = 0ms$
 - $n = 1'000 \rightarrow T(n \cdot \log_2) = 0ms$
 - $n = 10'000 \rightarrow T(n \cdot \log_2) = 2ms$
 - $n = 100'000 \rightarrow T(n \cdot \log_2) = 5ms$
 - $n = 1'000'000 \rightarrow T(n \cdot \log_2) = 19ms$
 - $n = 10'000'000 \rightarrow T(n \cdot \log_2) = 205ms$
 - $n = 100'000'000 \rightarrow T(n \cdot \log_2) = 2'301ms$
 - $n = 1'000'000'000 \rightarrow T(n \cdot \log_2) = 25'527ms$
 - $n = 10'000'000'000 \rightarrow T(n \cdot \log_2) = 298'345ms$

Når $n = 420'000'000$ tar det omtrent 10.49 sekunder for å fullføre iterasjonen. Så ikke veldig mye lavere enn dette.

Oppgave 2

- a) Denne oppgaven er referert til med kommentarer i Java koden (Linje 37 og 97).
- b) Her er en liste over tester på metoden *kubisk*(*n*) med økende problemstørrelse(*n*).
- $n = 10 \rightarrow T(n^3) = 0ms$
 - $n = 100 \rightarrow T(n^3) = 3ms$
 - $n = 1'000 \rightarrow T(n^3) = 493ms$
 - $n = 2'000 \rightarrow T(n^3) = 3'851ms$
 - $n = 2'500 \rightarrow T(n^3) = 7'647ms$
 - $n = 2'750 \rightarrow T(n^3) = 10'107ms$
 - $n = 3'000 \rightarrow T(n^3) = 12'978ms$

Når $n = 2'750$ tar det omtrent 10.107 sekunder for å fullføre iterasjonen. Så ikke veldig mye lavere enn dette.

Oppgave 3

- a) Denne oppgaven er referert til med kommentarer i Java koden (Linje 48 og 102).
- b) Her er en liste over tester på metoden *eksponentiell*(*n*) med økende problemstørrelse(*n*).
- $n = 10 \rightarrow T(2^n) = 0ms$
 - $n = 62 \rightarrow T(2^n) = 4746ms$
 - $n = 63 \rightarrow T(2^n) = 9536ms$
 - $n = 100 \rightarrow T(2^n) = 9489ms$
 - $n = 1'000 \rightarrow T(2^n) = 9488ms$
 - $n = 10'000 \rightarrow T(2^n) = 9496ms$
 - $n = 100'000 \rightarrow T(2^n) = 9541ms$

Her ser jeg at arbeidstiden treffer taket på 2^{63} . Jeg tror dette har med å gjøre at en *long* datatype har en maks verdi på $2^{63} - 1$. Dermed klarer ikke denne metoden å ta imot *n* verdier som er høyere enn 63 og dette bruker den ca 9.5 sekunder på å fullføre.

Oppgave 4

- a) Denne oppgaven er referert til med kommentarer i Java koden (Linje 58 og 107).
- b) Her er en liste over tester på metoden *kombinatorisk*(*n*) med økende problemstørrelse(*n*).
- $n = 10 \rightarrow T(n!) = 2ms$
 - $n = 15 \rightarrow T(n!) = 3ms$
 - $n = 20 \rightarrow T(n!) = 2'532ms$
 - $n = 25 \rightarrow T(n!) = 7'278ms$
 - $n = 26 \rightarrow T(n!) = 0ms$ - (Feiler)

Når *n!* får en høyere verdi en 25 så feiler iterasjonsprosessen. Maks arbeidstid for denne iterasjonene blir da for $n = 25!$ som tar omtrent 7.2 sekunder.